

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4

По дисциплине: «АППАРАТНО-ПРОГРАММНОЕ
ОБЕСПЕЧЕНИЕ ЭВМ И СЕТЕЙ»

Тема: «Архитектура и программирование
сопроцессора»

Выполнил:
Студент 4-го курса
Группы АС-66
Ануфриенко М. А.
Проверила:
Кулик А. Д.

Цель: изучение архитектуры и программной модели сопроцессора; практическая работа с системой целочисленных и вещественных команд.

1. Изучить следующие теоретические сведения: архитектура и программная модель сопроцессора – регистры и их назначение; форматы чисел, типовые команды, целочисленные команды (используя материалы лекций и методические указания).

2. Составить и отладить программу на языке ассемблера для вычисления

значения функции, используя регистры сопроцессора и целочисленные/вещественные команды согласно варианту, выданному преподавателем (таблица 2.2). Оформить ее в виде отдельной функции. Каждую ветку алгоритма оформить в виде отдельной внутренней процедуры. Ввод и вывод данных осуществляется через консоль.

Программу протестировать по всем условиям.

3. Подготовиться к защите

Содержание отчета по разделу 1:

- описание варианта задания;
- текст программы;
- скриншоты результатов.

Вариант 1

$$y = \begin{cases} 6xy - 4y, & x + y > 9; \\ \frac{2xy + 3 + x}{x^2 + 3y^2 + 1}, & x + y < -1; \\ 3x^2 - 2y + 6, & -1 \leq x + y \leq 9. \end{cases}$$

lab4_main.cpp

```
// lab4_main.cpp
// Лаб. №1 (сoproцессор). Раздел 1 (целочисленные команды) +
// Раздел 2 (вещественные команды). Вариант 1.
//
// y = 6xy - 4y,                x + y > 9
//   = (2xy + 3 + x) / (x^2 + 3y^2 + 1),  x + y < -1
//   = 3x^2 - 2y + 6,          -1 <= x + y <= 9

#include <stdio>
#include <windows.h>

extern "C" double ComputeIntAsm(int x, int y);           // реализация в lab4_int.asm
extern "C" double ComputeRealAsm(double x, double y);   // реализация в lab4_real.asm

int main()
{
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);

    printf("=== Раздел 1: целочисленные команды сопроцессора ===\n");
    printf("Введите целые x и y через пробел: ");
    int xi, yi;
    scanf_s("%d %d", &xi, &yi);
    double resultInt = ComputeIntAsm(xi, yi);
    printf("y = %.6f   (x=%d, y=%d, x+y=%d)\n\n", resultInt, xi, yi, xi + yi);
```

```

printf("=== Раздел 2: вещественные команды сопроцессора ===\n");
printf("Введите вещественные x и y через пробел: ");
double xr, yr;
scanf_s("%lf %lf", &xr, &yr);
double resultReal = ComputeRealAsm(xr, yr);
printf("y = %.6f (x=%.3f, y=%.3f, x+y=%.3f)\n", resultReal, xr, yr, xr + yr);

return 0;
}

```

lab4_int

```

; lab4_int.asm
; Лаб. №1 (сoproцессор), Раздел 1: целочисленные команды. Вариант 1.
;
; y = 6xy - 4y, x + y > 9
; = (2xy + 3 + x) / (x^2 + 3y^2 + 1), x + y < -1
; = 3x^2 - 2y + 6, -1 <= x + y <= 9
;
; extern "C" double ComputeIntAsm(int x, int y);
; x, y - целые числа; вычисление идёт через целочисленные команды
; сопроцессора (FILD/FIADD/FISUB/FIMUL - работают прямо с целым
; числом в памяти, без предварительного перевода в вещественный формат).

```

```

.386
.387
.MODEL FLAT, C
OPTION PROC:PRIVATE

```

```

.DATA
const_1 DWORD 1
const_2 DWORD 2
const_3 DWORD 3
const_4 DWORD 4
const_6 DWORD 6

```

```

.CODE

```

```

; -----
; Branch1: x + y > 9 => y = 6xy - 4y = y*(6x - 4)
; (внутренняя процедура - использует [ebp+8]/[ebp+12] того кадра
; стека, что уже подготовлен вызывающей ComputeIntAsm)
; -----

```

```

Branch1 PROC
    fild dword ptr [ebp+8]      ; ST0 = x
    fimul const_6              ; ST0 = 6x
    fisub const_4              ; ST0 = 6x - 4
    fimul dword ptr [ebp+12]    ; ST0 = (6x-4)*y = 6xy - 4y
    ret
Branch1 ENDP

```

```

; -----
; Branch2: x + y < -1 => y = (2xy+3+x) / (x^2+3y^2+1)
; числитель переписан как x*(2y+1)+3 - алгебраически то же самое
; -----

```

```

Branch2 PROC
    ; числитель
    fild dword ptr [ebp+12]    ; ST0 = y
    fimul const_2              ; ST0 = 2y
    fiadd const_1              ; ST0 = 2y + 1
    fimul dword ptr [ebp+8]    ; ST0 = x*(2y+1)
    fiadd const_3              ; ST0 = числитель

    ; знаменатель
    fild dword ptr [ebp+8]      ; ST0 = x, ST1 = числитель
    fimul dword ptr [ebp+8]    ; ST0 = x^2, ST1 = числитель
    fild dword ptr [ebp+12]    ; ST0 = y, ST1 = x^2, ST2 = числитель
    fimul dword ptr [ebp+12]    ; ST0 = y^2, ST1 = x^2, ST2 = числитель

```

```

    fimul const_3                ; ST0 = 3y^2, ST1 = x^2, ST2 = числитель
    faddp st(1), st(0)           ; ST0 = x^2+3y^2,      ST1 = числитель
    fiadd const_1                ; ST0 = знаменатель,    ST1 = числитель

    fdivp st(1), st(0)           ; ST0 = числитель / знаменатель
    ret
Branch2 ENDP

```

```

; -----
; Branch3: -1 <= x + y <= 9  =>  y = 3x^2 - 2y + 6
; -----

```

```

Branch3 PROC
    fild dword ptr [ebp+8]        ; ST0 = x
    fimul dword ptr [ebp+8]      ; ST0 = x^2
    fimul const_3                ; ST0 = 3x^2

    fild dword ptr [ebp+12]      ; ST0 = y,    ST1 = 3x^2
    fimul const_2                ; ST0 = 2y,    ST1 = 3x^2

    fsubp st(1), st(0)           ; ST0 = 3x^2 - 2y
    fiadd const_6                ; ST0 = 3x^2 - 2y + 6
    ret
Branch3 ENDP

```

```

; -----
; double ComputeIntAsm(int x, int y)
; Диспетчер: сравнивает x+y с границами и вызывает нужную ветку.
; Результат остаётся в ST(0) - так x87/cdec1 возвращает double.
; -----

```

```

PUBLIC ComputeIntAsm
ComputeIntAsm PROC
    push ebp
    mov  ebp, esp

    mov  eax, [ebp+8]            ; x
    add  eax, [ebp+12]           ; eax = x + y (обычное целочисленное сложение, FPU тут не
    ; нужен)

```

```

    cmp  eax, 9
    jg   DoBranch1              ; x+y > 9

```

```

    cmp  eax, -1
    jl   DoBranch2              ; x+y < -1

```

```

    call Branch3                ; иначе: -1 <= x+y <= 9
    jmp  ComputeDone

```

```

DoBranch1:
    call Branch1
    jmp  ComputeDone

```

```

DoBranch2:
    call Branch2

```

```

ComputeDone:
    pop  ebp
    ret
ComputeIntAsm ENDP
END

```

lab4_real

```
; lab4_real.asm
; Лаб. №1 (сопроцессор), Раздел 2: вещественные команды. Вариант 1.
;
;  $y = 6xy - 4y$ ,  $x + y > 9$ 
;  $= (2xy + 3 + x) / (x^2 + 3y^2 + 1)$ ,  $x + y < -1$ 
;  $= 3x^2 - 2y + 6$ ,  $-1 \leq x + y \leq 9$ 
;
; extern "C" double ComputeRealAsm(double x, double y);
; x, y - вещественные числа; вычисление идёт через "чистые"
; вещественные команды сопроцессора (FLD/FADD/FSUB/FMUL/FDIV,
; без буквы I - операнды уже в формате с плавающей точкой).

.386
.387
.MODEL FLAT, C
OPTION PROC:PRIVATE

.DATA
const_1 REAL8 1.0
const_2 REAL8 2.0
const_3 REAL8 3.0
const_4 REAL8 4.0
const_6 REAL8 6.0
const_9 REAL8 9.0
const_neg1 REAL8 -1.0

.CODE

; -----
; Branch1:  $x + y > 9 \Rightarrow y = 6xy - 4y = y(6x - 4)$ 
; double-параметры в cdec1 занимают по 8 байт: x -> [ebp+8], y -> [ebp+16]
; -----
Branch1 PROC
    fld qword ptr [ebp+8]          ; ST0 = x
    fmul const_6                  ; ST0 = 6x
    fsub const_4                  ; ST0 = 6x - 4
    fmul qword ptr [ebp+16]       ; ST0 = (6x-4)*y = 6xy - 4y
    ret
Branch1 ENDP

; -----
; Branch2:  $x + y < -1 \Rightarrow y = (2xy+3+x) / (x^2+3y^2+1)$ 
; числитель переписан как  $x*(2y+1)+3$ 
; -----
Branch2 PROC
    ; числитель
    fld qword ptr [ebp+16]        ; ST0 = y
    fmul const_2                  ; ST0 = 2y
    fadd const_1                  ; ST0 = 2y + 1
    fmul qword ptr [ebp+8]        ; ST0 = x*(2y+1)
    fadd const_3                  ; ST0 = числитель

    ; знаменатель
    fld qword ptr [ebp+8]         ; ST0 = x, ST1 = числитель
    fmul qword ptr [ebp+8]        ; ST0 = x^2, ST1 = числитель
    fld qword ptr [ebp+16]        ; ST0 = y, ST1 = x^2, ST2 = числитель
    fmul qword ptr [ebp+16]       ; ST0 = y^2, ST1 = x^2, ST2 = числитель
    fmul const_3                  ; ST0 = 3y^2, ST1 = x^2, ST2 = числитель
    faddp st(1), st(0)            ; ST0 = x^2+3y^2, ST1 = числитель
    fadd const_1                  ; ST0 = знаменатель, ST1 = числитель

    fdivp st(1), st(0)            ; ST0 = числитель / знаменатель
    ret
Branch2 ENDP
```

```

; -----
; Branch3: -1 <= x + y <= 9  =>  y = 3x^2 - 2y + 6
; -----
Branch3 PROC
    fld qword ptr [ebp+8]          ; ST0 = x
    fmul qword ptr [ebp+8]        ; ST0 = x^2
    fmul const_3                  ; ST0 = 3x^2

    fld qword ptr [ebp+16]        ; ST0 = y,    ST1 = 3x^2
    fmul const_2                  ; ST0 = 2y,    ST1 = 3x^2

    fsubp st(1), st(0)             ; ST0 = 3x^2 - 2y
    fadd const_6                  ; ST0 = 3x^2 - 2y + 6
    ret
Branch3 ENDP

; -----
; double ComputeRealAsm(double x, double y)
; Диспетчер: x,y вещественные, поэтому сравнение x+y с границами
; идёт через сам сопроцессор (fcomp), а не через обычный cmp.
; -----
PUBLIC ComputeRealAsm
ComputeRealAsm PROC
    push ebp
    mov  ebp, esp

    ; Первая проверка: x + y > 9 ?
    fld qword ptr [ebp+8]          ; ST0 = x
    fadd qword ptr [ebp+16]        ; ST0 = x + y
    fcomp const_9                 ; сравнить ST0 с 9.0, вытолкнуть ST0 (стек снова пуст)
    fnstsw ax                     ; перенести статус-слово FPU в AX
    sahf                          ; AH -> флаги процессора, теперь работают обычные
Jcc
    ja  DoBranch1                  ; x+y > 9

    ; Вторая проверка: x + y < -1 ?
    fld qword ptr [ebp+8]          ; пересчитываем сумму заново - стек FPU уже чист после
fcomp
    fadd qword ptr [ebp+16]
    fcomp const_neg1
    fnstsw ax
    sahf
    jb  DoBranch2                  ; x+y < -1

    call Branch3                   ; иначе: -1 <= x+y <= 9
    jmp ComputeDone

DoBranch1:
    call Branch1
    jmp ComputeDone

DoBranch2:
    call Branch2

ComputeDone:
    pop  ebp
    ret
ComputeRealAsm ENDP

END

```

```
Microsoft Visual Studio Debug Console
=== Раздел 1: целочисленные команды сопроцессора ===
Введите целые x и y через пробел: 12341 343
y = 25396406.000000 (x=12341, y=343, x+y=12684)

=== Раздел 2: вещественные команды сопроцессора ===
Введите вещественные x и y через пробел: 12341 343
y = 25396406.000000 (x=12341.000, y=343.000, x+y=12684.000)

C:\Users\maksim\source\repos\lab4\Debug\lab4.exe (process 6724) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .

=== Раздел 1: целочисленные команды сопроцессора ===
Введите целые x и y через пробел: 2 3
y = 12.000000 (x=2, y=3, x+y=5)

=== Раздел 2: вещественные команды сопроцессора ===
Введите вещественные x и y через пробел: 2.00 3.000
y = 12.000000 (x=2.000, y=3.000, x+y=5.000)

C:\Users\maksim\source\repos\lab4\Debug\lab4.exe (process 1868) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .

=== Раздел 1: целочисленные команды сопроцессора ===
Введите целые x и y через пробел: -123
-45
y = 0.516388 (x=-123, y=-45, x+y=-168)

=== Раздел 2: вещественные команды сопроцессора ===
Введите вещественные x и y через пробел: -123.00 -45.00
y = 0.516388 (x=-123.000, y=-45.000, x+y=-168.000)

C:\Users\maksim\source\repos\lab4\Debug\lab4.exe (process 3240) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Вывод: В ходе выполнения лабораторной работы изучена архитектура и программная модель сопроцессора, реализована функция вычисления кусочно-заданного выражения (Вариант 1) с использованием двух систем команд: целочисленной (FILD/FIADD/FISUB/FIMUL) в Разделе 1 и вещественной (FLD/FADD/FSUB/FMUL/FDIV) в Разделе 2. Каждая из трёх ветвей алгоритма, соответствующих условиям на $x+y$, оформлена отдельной внутренней процедурой, вызываемой из общей диспетчерской функции после сравнения суммы $x+y$ с границами диапазонов.

Работоспособность программы проверена на трёх наборах входных данных, последовательно покрывающих все условия варианта: $x=12341, y=343$ ($x+y=12684 > 9$, ветвь $6xy-4y$), $x=2, y=3$ ($-1 \leq x+y \leq 9$, ветвь $3x^2-2y+6$), $x=-123, y=-45$ ($x+y=-168 < -1$, ветвь деления). Во всех трёх случаях результаты Раздела 1 (целые входные данные) и Раздела 2 (вещественные входные данные) совпали между собой и с результатом ручного расчёта по формуле, что подтверждает корректность реализации обеих систем команд сопроцессора.